

5BUIS018W Business Development Coursework

Student ID: w1967351

Website link: <https://w1967351.users.ecs.westminster.ac.uk/bis/public/index.php>

A6 documentation (300 words)

Development Process: Initial Steps and Sequence.

The project began by setting up the development of two distinct databases: one for **Activities** and one for **Training**. This stage was key, because it confirmed the structure for how the website would store and retrieve data. After developing the databases in phpMyAdmin, I focused on creating a reliable connection between the database and the website. This was done by building a **Database** class, which made it easier to work with the database.

Next, I built the basic framework of the website. The front end had a logo in the upper-right corner, a red coloured font title at the top and a sidebar for easy navigation. The primary material was put in the centre to keep the layout simple and user-friendly. I also included important links such as View Activities and Training to assist people in navigating the site easier.

For administrative responsibilities, I concentrated on developing capabilities that allow administrators to **add**, **view**, and **update** records in the **Training** database. I utilised simple HTML forms to make it quick and simple to add data into the system.

Progression: Testing and Refinement

In the testing phase, I worked on validating the functionality of database interactions. This included making certain that data was correctly retrieved and updated in the database. I also worked on refining the error-handling mechanism, especially through the **Database class**, ensuring any issues with database connections or queries would be captured and appropriately managed.

Program Structure

The application follows a well-organised structure:

The **Private Folder** contains all of the essential backend files, including the Database, Activities, and Training courses. It also includes setup scripts, database functions, and credentials—basically everything that keeps the site functioning properly.

The **Public Folder** has user-interactive pages, such as forms for record Add, View, Browse and Update, as well as Training and Activities.

This structured approach ensured that the development was manageable and that the application was built incrementally, tested, and refined at each stage.

B1) Display list of Activities

Above is the code that implements the "Display List of Activities" feature. The code retrieves all activities from the database and displays them in a table format.

The implementation begins with the placement of the **initialize.php** file, which configures the database connection and other settings. The **activities::find_all()** method retrieves all activity entries from the database. The method returns a list of activity objects, each with characteristics such as **ID**, **Name**, **Description**, **Benefits**, and **Price**. The implementation begins with the inclusion of the **initialize.php** file, which is most likely responsible for configuring the database connection and other settings. The **activities::find_all()** function returns all activity entries from the database. It returns a list of activity objects, each with attributes such as **ID**, **Name**, **Description**, **Benefits**, and **Price**. The data is then shown in an HTML table, with each column representing a particular activity attribute and each row corresponding to a specific action. The loop displays each activity's **ID**, **Name**, **Description**, **Benefits**, and **Price**, as well as a link to a different page with further information. The link creates a unique URL for each action, allowing people to access further information about it. The code adheres to object-oriented concepts by using the activities class to encapsulate activity-related attributes and methods, such as finding all activities with **find_all()**. This method abstracts the database interaction, allowing the display logic to remain clean and focused on the presentation. The usage of classes and objects makes the system more versatile and simple to maintain. The SQL query is hidden within the class function, keeping the database activities out of sight. This simplifies the code and makes it easier to understand, focussing only on what needs to be presented. Thus, the object-oriented approach promotes encapsulation, reusability, and data abstraction, which enhances code organisation and maintainability.

B2) Add new Training

```
<?php
require_once
'../private/initialize.php';//initialize the web site
?>

<!DOCTYPE html>
<html lang="en">
<head>
<title>CSS Template</title>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet" type="text/css" href="style.css">
</head>

<body>
<div class="header">
ActiveAtHome <br>
Add - update - view activities and trainers in training <br>
</div>

<div class="row">
  <div class="column side">
    <?php include 'navigation.html';?>
  </div>
  <div class="column middle">

<?php

if($_SERVER['REQUEST_METHOD'] === 'POST') {

$args=[];//in $args array we collect all the new values
$args['name']=$_POST['name'];
$args['email']=$_POST['email'];
$args['location']=$_POST['location'];
$args['certifications']=$_POST['certifications'];
$args['years']=$_POST['years'];
$args['specialization']=$_POST['specialization'];

$training = new Training;// we create a new object with the values we just entered
$training->Name = $args["name"];
$training->Email = $args["email"];
```

```
$training->Location = $args["location"];
$training->Certifications = $args["certifications"];
$training->Years = $args["years"];
$training->Specialization = $args["specialization"];
```

```
$results = $training->create();//
```

```
if($results){
echo "New Record added successfully";
}
```

```
} else {
```

```
echo " <p> Use the following form to enter details for the new item (record / object) <br> ";
echo "ATTENTION: all *** fields must be numbers </p>";
```

```
echo "<form action=add_training.php method='post'>";
echo "<table>";
echo "<tr> <td> Name </td> <td> <input type='text' name ='name'> </td> </tr>";
echo "<tr> <td> Email </td> <td> <input type='text' name ='email'> </td> </tr>";
echo "<tr> <td> Location *** </td> <td> <input type='text' name ='location'> </td> </tr>";
echo "<tr> <td> Certifications </td> <td> <input type='text' name ='certifications'> </td> </tr>";
echo "<tr> <td> Years </td> <td> <textarea name='years' rows='5' cols='30'> </textarea> </td>
</tr>";
echo "<tr> <td> Specialization *** </td> <td> <input type='text' name ='specialization'> </td>
</tr>";
echo "</table>";
echo " <br> <br> <input type='submit' value='Add New' />";
echo "</form>";
```

```
}
```

```
?>
```

```
</div>
```

```
<div class="footer">
  BIS Development Module<br>
  Not real site
</div>
```

</body>

</html>

The "Add New Training" function is handled by the **add_training.php** code, which accepts user input via a form. When the form is submitted, the data (**Name**, **Email**, **Location**, **Certifications**, **Years**, and **Specialisation**) is collected via the **\$_POST** method and added to a new Training object containing the trainer's information. The **create()** function is then used on this object to put the new trainer into the database. While the SQL query for insertion isn't visible directly in the source, it's abstracted within the **create()** function of the Training class that produces and executes an **INSERT INTO** SQL statement. If the insertion is successful, a success message is displayed. If the form hasn't been submitted, the HTML form for entering trainer details is shown. This implementation follows object-oriented principles by using the Training class to encapsulate the trainer data and SQL logic, promoting reusability, modularity, and abstraction. The class manages all database interactions, separating the difficult SQL complexities from the rest of the application. This simplifies code maintenance and expansion because the class contains everything relevant to the Training object. This ensures that any modifications to the database logic or structure do not have an impact on the rest of the software. The object-oriented approach also helps to maintain the code clean and organised, which is essential for scalability and debugging.

References

Week 8 to week 11 tutorial lessons

Appendix A

Private folder codes

Class activities

<?php

```
class Activities {

    static public $database;

    static public function set_database($database) {
        self::$database = $database;
    }

    static public function find_by_sql($sql) {
        $results = self::$database->query($sql);
        if (!$results) { // add evaluation of the query succeeded or failed
            exit("Database query failed.");
        }
        if ($results) {
            echo "Database query succeeded. <br>";
        }
        $object_array = [];
        while ($record = $results->fetch_assoc()) { // get the first row as an array and create an
object
            $object_array[] = self::instantiate($record);
        }
        return $object_array;
    }

    static public function find_all() {
        $sql = "SELECT * FROM Activities";
        return self::find_by_sql($sql);
    }

    static public function find_by_id($id) {
        $sql = "SELECT * FROM Activities WHERE id=" . self::$database->escape_string($id) . """;
// escape the string
        $result = self::find_by_sql($sql);
        if (!empty($result)) {
```

```

        echo "not empty";
        return array_shift($result);
    } else {
        echo "empty";
    }
}

static protected function instantiate($record) {
    $object = new Activities;
    foreach ($record as $property => $value) {
        if (property_exists($object, $property)) {
            $object->$property = $value;
        }
    }
    return $object;
}

```

```

public $ID;
public $Name;
public $Description;
public $Benefits;
public $Price;
}

```

?>

Class database

<?php

```

class Database {

    static protected $database;
    static protected $table_name = "";
    static protected $columns;

    static public function set_database($database) {
        self::$database = $database;
    }

    static public function find_by_sql($sql){

```

```

        //echo $sql;
        $results = self::$database->query($sql);
        if(!$results) {
//add valuation of the query succeeded or failed
            exit("Database query failed. ");
        }
        $object_array=[];
        while($record = $results->fetch_assoc()) { //get the first row as an array
and create an object
            $object_array[] = static::instantiate($record);
        }
        return $object_array;
    }

```

```

    static public function find_all() {
        $sql = "SELECT * FROM " . static::$table_name; //why not
self::$table_name???
        return static::find_by_sql($sql);
    }

```

```

    static public function find_by_id($id) {
        $sql = "SELECT * FROM " . static::$table_name . " WHERE id=" .
static::$database->escape_string($id) . ""'; //escape the string
        $result = static::find_by_sql($sql);
        if(!empty($result)){
            return array_shift($result);
        } else {
            echo "empty";
        }
    }

```

```

    static protected function instantiate($record) {
        $object = new static;
        foreach($record as $property => $value) {
            if(property_exists($object, $property)){
                $object->$property=$value;
            }
        }
        return $object;
    }

```

```

    static public function verify_user($loginname, $password){

```

```

        $sql = "SELECT * FROM " . static::$table_name . " WHERE
loginname='$loginname' AND password='$password' "; //escape the string
        $result = static::find_by_sql($sql);
        return array_shift($result);
    }

```

```

public function update() {
    $attributes = $this->attributes();
    $attribute_pairs=[];
    foreach($attributes as $key=>$value) {
        $attribute_pairs[] = "{$key}='{$value}'";
    }
    $sql = "UPDATE " . static::$table_name . " SET ";
    $sql .= join(', ', $attribute_pairs);
    $sql .= " WHERE id=" . static::$database->escape_string($this->id) . " "; //
    $results = static::$database->query($sql); //
    return $results;
}

```

```

public function create() {
    $attributes = $this->attributes();
    $sql = "INSERT INTO " . static::$table_name . " (";
    $sql .= join(', ', array_keys($attributes));
    $sql .= ") VALUES (";
    $sql .= join("'", array_values($attributes));
    $sql .= "' )";
    $results = static::$database->query($sql);
    if($results) {
        $this->id = static::$database->insert_id;
    }
    return $results;
}

```

```

public function delete() {
    $sql = "DELETE FROM " . static::$table_name . " WHERE id=" . $this->id . "
LIMIT 1";
    $results = static::$database->query($sql);
    return $results;
}

```

// \$attributes are the properties that have the database columns excluding 'id'

```

public function attributes() {
    $attributes=[];
    foreach(static::$db_columns as $column) {
        if($column == 'id') { continue; }
        $attributes[$column] = $this->$column;
    }
    return $attributes;
}

public function merge_attributes($args=[]) {
    foreach($args as $key=>$value) {
        if(property_exists($this, $key)) {
            $this->$key = $value;
        }
    }
}

```

```

}

```

```

?>

```

Class training

```

<?php

```

```

class Training {

    static public $database;

    static public function set_database($database) {
        self::$database = $database;
    }

    static public function find_by_sql($sql) {
        $results = self::$database->query($sql);

        if (!$results) {
            exit("Database query failed.");
        }
    }
}

```

```

        echo "Database query succeeded. <br>";

        $object_array = [];
        while ($record = $results->fetch_assoc()) {
            $object_array[] = self::instantiate($record);
        }
        return $object_array;
    }

    static public function find_all() {
        $sql = "SELECT * FROM Training";
        return self::find_by_sql($sql);
    }

    static public function find_by_id($id) {
        $sql = "SELECT * FROM Training WHERE id=" . self::$database->escape_string($id) . "";
        $result = self::find_by_sql($sql);
        if (!empty($result)) {
            return array_shift($result);
        } else {
            return null;
        }
    }

    static protected function instantiate($record) {
        $object = new Training;
        foreach ($record as $property => $value) {
            if (property_exists($object, $property)) {
                $object->$property = $value;
            }
        }
        return $object;
    }

    // Fixed create() method
    public function create() {
        $sql = "INSERT INTO Training (Name, Email, Location, Certifications, Years,
Specialization) VALUES (" .
            self::$database->escape_string($this->Name) . ", " .
            self::$database->escape_string($this->Email) . ", " .
            self::$database->escape_string($this->Location) . ", " .
            self::$database->escape_string($this->Certifications) . ", " .
            self::$database->escape_string($this->Years) . ", " .

```

```

        self::$database->escape_string($this->Specialization) . "''");

        $result = self::$database->query($sql);
        return $result ? true : false;
    }

    // New update() method
    public function update() {
        // SQL query to update the existing record in the database
        $sql = "UPDATE Training SET
            Name = '' . self::$database->escape_string($this->Name) . '',
            Email = '' . self::$database->escape_string($this->Email) . '',
            Location = '' . self::$database->escape_string($this->Location) . '',
            Certifications = '' . self::$database->escape_string($this->Certifications) . '',
            Years = '' . self::$database->escape_string($this->Years) . '',
            Specialization = '' . self::$database->escape_string($this->Specialization) . ''
            WHERE ID = '' . self::$database->escape_string($this->ID) . ''";

        $result = self::$database->query($sql);
        return $result ? true : false;
    }

    // Object properties
    public $ID;
    public $Name;
    public $Email;
    public $Location;
    public $Certifications;
    public $Years;
    public $Specialization;
}

```

?>

Credentials

```

<?php
// Credentials of my db set <br>' ;

//Keep database details in a seperate file
define("DB_SERVER", "phpmyadmin.ecs.westminster.ac.uk");
define("DB_USER", "w1967351");           //your login name
define("DB_PASS", "f5XsqKAq8Q9C");       //your password. You can find it in the address:
https://support.ecs.westminster.ac.uk/mysql/index.php

```

```
define("DB_NAME", "w1967351_0");           //your database
```

```
?>
```

database functions

```
<?php
```

```
function db_connect() {  
    $database = new mysqli("phpmyadmin.ecs.westminster.ac.uk", "w1967351",  
"f5XsqKAq8Q9C", database: "w1967351_0"); //create the object $connection  
    confirm_db_connect($database);  
    return $database; // the function returns the object $connection  
}
```

```
function confirm_db_connect($database) {  
  
    if($database->connect_errno==0) {  
        echo "Connection to DB succeed. <br>";  
    }  
  
    if($database->connect_errno>0) {  
        $msg = "Database connection failed: ";  
        $msg .= $database->connect_error;  
        $msg .= " (" . $database->connect_errno . ")";  
        echo $msg;  
        echo "<br>";  
    }  
}
```

```
?>
```

Initialize

```
<?php
```

```
require_once('credentials.php');  
require_once('database_functions.php');  
require_once('class_activities.php');  
require_once('class_training.php');
```

```
$database = db_connect(); // call the connection;  
Activities::set_database($database); // class 'becomes aware' about the database
```

```
Training::set_database($database); // class 'becomes aware' about the database
```

```
?>
```

Public folder codes

Add Training

```
<?php
require_once
'../private/initialize.php';//initialize the web site
?>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>CSS Template</title>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet" type="text/css" href="style.css">
</head>
```

```
<body>
<div class="header">
ActiveAtHome <br>
Add - update - view activities and trainers in training <br>
</div>
```

```
<div class="row">
  <div class="column side">
    <?php include 'navigation.html';?>
  </div>
  <div class="column middle">
```

```
<?php
```

```
if($_SERVER['REQUEST_METHOD'] === 'POST') {
```

```
$args=[];//in $args array we collect all the new values
$args['name']=$_POST['name'];
$args['email']=$_POST['email'];
$args['location']=$_POST['location'];
$args['certifications']=$_POST['certifications'];
$args['years']=$_POST['years'];
```

```
$args['specialization']=$_POST['specialization'];
```

```
$training = new Training;// we create a new object with the values we just entered
```

```
$training->Name = $args["name"];
```

```
$training->Email = $args["email"];
```

```
$training->Location = $args["location"];
```

```
$training->Certifications = $args["certifications"];
```

```
$training->Years = $args["years"];
```

```
$training->Specialization = $args["specialization"];
```

```
$results = $training->create();//
```

```
if($results){
```

```
echo "New Record added successfully";
```

```
}
```

```
} else {
```

```
echo " <p> Use the following form to enter details for the new item (record / object) <br> ";
```

```
echo "ATTENTION: all *** fields must be numbers </p>";
```

```
echo "<form action=add_training.php method='post'>";
```

```
echo "<table>";
```

```
echo "<tr> <td> Name </td> <td> <input type='text' name ='name'> </td> </tr>";
```

```
echo "<tr> <td> Email </td> <td> <input type='text' name ='email'> </td> </tr>";
```

```
echo "<tr> <td> Location *** </td> <td> <input type='text' name ='location'> </td> </tr>";
```

```
echo "<tr> <td> Certifications </td> <td> <input type='text' name ='certifications'> </td> </tr>";
```

```
echo "<tr> <td> Years </td> <td> <textarea name='years' rows='5' cols='30'> </textarea> </td> </tr>";
```

```
echo "<tr> <td> Specialization *** </td> <td> <input type='text' name ='specialization'> </td> </tr>";
```

```
echo "</table>";
```

```
echo " <br> <br> <input type='submit' value='Add New' />";
```

```
echo "</form>";
```

```
}
```

```
?>
```

```
</div>
```

```
<div class="footer">
  BIS Development Module<br>
  Not real site
</div>
```

```
</body>
```

```
</html>
```

Update Training

```
<?php
require_once '../private/initialize.php'; //initialize the web site
?>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>CSS Template</title>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet" type="text/css" href="style.css">
</head>
```

```
<body>
<div class="header">
ActiveAtHome <br>
Add - Update - View Activities and Trainers in Training
</div>
```

```
<div class="row">
  <div class="column side">
    <?php include 'navigation.html';?>
  </div>
  <div class="column middle">
```

```
<?php
// Ensure 'id' is passed in the URL
if (isset($_GET['id'])) {
  $id = $_GET['id'];
} else {
  // Handle the case where 'id' is not provided in the URL
```

```

    echo "Error: ID not provided.";
    exit; // or redirect to another page
}

echo "my id is: " . $id . "<br>";

// Find the specific training record by ID
$specific_training = Training::find_by_id($id); // Call the find_by_id() function to find the record

// Check if the training record is found
if ($specific_training === null) {
    echo "Error: No training record found for ID: " . $id;
    exit; // Or redirect the user to another page
}

// Start checking for POST request
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // Directly assign values from the form to the object's properties
    $specific_training->Name = $_POST['name'];
    $specific_training->Email = $_POST['email'];
    $specific_training->Location = $_POST['location'];
    $specific_training->Certifications = $_POST['certifications'];
    $specific_training->Years = $_POST['years'];
    $specific_training->Specialization = $_POST['specialization'];

    // Update the record in the database
    $results = $specific_training->Update();

    // Check if the update was successful
    if ($results) {
        echo "Successful updating";
    } else {
        echo "Error: Could not update the record.";
    }
}

// Show updated record details
echo "<p> Details of the updated item </p> ";

echo "<table>";
echo "<tr> <td> <b> ID </b> </td> <td>" . $specific_training->ID . "</td> </tr>";
echo "<tr> <td> <b> Name </b> </td> <td>" . $specific_training->Name . "</td> </tr>";
echo "<tr> <td> <b> Email </b> </td> <td>" . $specific_training->Email . "</td> </tr>";
echo "<tr> <td> <b> Location </b> </td> <td>" . $specific_training->Location . "</td> </tr>";

```

```

        echo "<tr> <td> <b> Certifications </b> </td> <td>" . $specific_training->Certifications . "</td>
</tr>";
        echo "<tr> <td> <b> Years </b> </td> <td>" . $specific_training->Years . "</td> </tr>";
        echo "<tr> <td> <b> Specialization </b> </td> <td>" . $specific_training->Specialization .
"</td> </tr>";
        echo "</table>";
    } else {
        // If there is no POST action, present the values of the record in a form to be updated
        echo " <p> Use the following form to update the selected item <br> ";
        echo "ATTENTION: all *** fields must be numbers </p>";

        echo "<form action='update_training.php?id=" . $id . "' method='post'>";
        echo "<table>";
        echo "<tr> <td> Name </td> <td> <input type='text' name='name' value='" .
htmlspecialchars($specific_training->Name) . "'> </td> </tr>";
        echo "<tr> <td> Email </td> <td> <input type='text' name='email' value='" .
htmlspecialchars($specific_training->Email) . "'> </td> </tr>";
        echo "<tr> <td> Location *** </td> <td> <input type='text' name='location' value='" .
htmlspecialchars($specific_training->Location) . "'> </td> </tr>";
        echo "<tr> <td> Certifications </td> <td> <input type='text' name='certifications' value='" .
htmlspecialchars($specific_training->Certifications) . "'> </td> </tr>";
        echo "<tr> <td> Years </td> <td> <textarea name='years' rows='5' cols='30'>" .
htmlspecialchars($specific_training->Years) . "</textarea> </td> </tr>";
        echo "<tr> <td> Specialization *** </td> <td> <input type='text' name='specialization' value='" .
htmlspecialchars($specific_training->Specialization) . "'> </td> </tr>";
        echo "</table>";
        echo " <input type='submit' value='Update Record' />";
        echo "</form>";
    }
?>

</div>

<div class="footer">
    BIS Development Module<br>
    Not real site
</div>

</body>

</html>

```

Delete Training

```
<?php
require_once '../private/initialize.php'; // Ensure initialization
require_once '../private/class_training.php'; // Include Training class

// Ensure ID is provided
if (!isset($_GET['id']) || empty($_GET['id'])) {
    die("Error: No ID provided.");
}

$id = intval($_GET['id']); // Convert ID to integer for security

// Find the training record by ID
$delete_training = Training::find_by_id($id);

if ($delete_training) {
    // Call delete method
    if ($delete_training->delete()) {
        echo "<p>The following entry has been deleted:</p>";
        echo "<table border='1'>";
        echo "<tr><td><b>ID</b></td><td>" . htmlspecialchars($delete_training->id) . "</td></tr>";
        echo "<tr><td><b>Name</b></td><td>" . htmlspecialchars($delete_training->Name) .
"</td></tr>";
        echo "<tr><td><b>Email</b></td><td>" . htmlspecialchars($delete_training->Email) .
"</td></tr>";
        echo "<tr><td><b>Location</b></td><td>" . htmlspecialchars($delete_training->Location) .
"</td></tr>";
        echo "<tr><td><b>Certifications</b></td><td>" .
htmlspecialchars($delete_training->Certifications) . "</td></tr>";
        echo "<tr><td><b>Years</b></td><td>" . htmlspecialchars($delete_training->years) .
"</td></tr>";
        echo "<tr><td><b>Specialization</b></td><td>" .
htmlspecialchars($delete_training->Specialization) . "</td></tr>";
        echo "</table>";
    } else {
        echo "<p>Error: Could not delete training entry.</p>";
    }
} else {
    echo "<p>Error: Training entry not found.</p>";
}
```

?>

<!DOCTYPE html>

<html lang="en">

<head>

 <title>ActiveAtHome - Delete Training</title>

 <meta charset="utf-8">

 <meta name="viewport" content="width=device-width, initial-scale=1">

 <link rel="stylesheet" type="text/css" href="style.css">

</head>

<body>

<div class="header">

 ActiveAtHome

 Add - update - view activities and trainers in training

</div>

<div class="row">

 <div class="column side">

 <?php include 'navigation.html'; ?> <!-- Navigation inclusion -->

 </div>

 <div class="column middle">

 Back to Training List

 </div>

</div>

<div class="footer">

 BIS Design & Development Module

 Not a real site

</div>

</body>

</html>

View (for Activities)

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>CSS Template</title>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet" type="text/css" href="style.css">
</head>

<body>
<div class="header">
ActiveAtHome <br>
Add - Update - View Activities and Trainers in Training
</div>

<div class="row">
  <div class="column side">
    <?php include 'navigation.html';?>
  </div>
  <div class="column middle">

<?php
include '../private/initialize.php';//initialize the web site
include_once '../private/class_activities.php'; // include the Activities class

$id = $_GET['id'];

if(!$id) {
echo "problem - redirect";
} else {
echo $id;
}

$specific_activities = Activities::find_by_id($id);//call the find_by_id() function

echo " <p> Details of the selected item </p> ";

echo "<table>";
echo "<tr> <td><b> id </b> </td> <td>" . $specific_activities->ID . "</td> </tr>";
```

```

echo "<tr> <td> <b> Name </b> </td> <td>" . $specific_activities->Name . "</td> </tr>";
echo "<tr> <td> <b> Description </b> </td> <td>" . $specific_activities->Description . "</td>
</tr>";
echo "<tr> <td> <b> Benefits </b> </td> <td>" . $specific_activities->Benefits . "</td> </tr>";
echo "<tr> <td> <b> price </b> </td> <td>" . $specific_activities->Price . "</td> </tr>";
echo "</table>";

?>

```

```

</div>
</div>

```

```

<div class="footer">
    BIS Design & Development Module<br>
    Not real site
</div>

```

```

</body>
</html>

```

View Training

```

<!DOCTYPE html>
<html lang="en">
<head>
<title>CSS Template</title>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet" type="text/css" href="style.css">
</head>

```

```

<body>
<div class="header">
ActiveAtHome <br>
Add - Update - View Activities and Trainers in Training
</div>

```

```

<div class="row">
    <div class="column side">
        <?php include 'navigation.html';?>
    </div>
    <div class="column middle">

```

```

<?php

```

```
include '../private/initialize.php';//initialize the web site
```

```
$id = $_GET['id'];
```

```
if(!$id) {  
    echo "problem - redirect";  
} else {  
    echo $id;  
}
```

```
$specific_Training = Training::find_by_id($id);//call the find_by_id() function
```

```
echo "<p> Details of the selected item </p> ";
```

```
echo "<table>";  
echo "<tr> <td><b> id </b> </td> <td>" . $specific_Training->ID . "</td> </tr>";  
echo "<tr> <td> <b> Name </b> </td> <td>" . $specific_Training->Name . "</td> </tr>";  
echo "<tr> <td> <b> Email </b> </td> <td>" . $specific_Training->Email . "</td> </tr>";  
echo "<tr> <td> <b> Location </b> </td> <td>" . $specific_Training->Location . "</td> </tr>";  
echo "<tr> <td> <b> Certifications </b> </td> <td>" . $specific_Training->Certifications . "</td>  
</tr>";  
echo "<tr> <td> <b> Years </b> </td> <td>" . $specific_Training->Years . "</td> </tr>";  
echo "<tr> <td> <b> Specialization </b> </td> <td>" . $specific_Training->Specialization . "</td>  
</tr>";  
echo "</table>";
```

```
?>
```

```
</div>  
</div>
```

```
<div class="footer">  
    BIS Design & Development Module<br>  
    Not real site  
</div>
```

```
</body>  
</html>
```

Navigation

```
<p> Select one of the following <p>
<ul>
  <li> <a href ="index.php"> Home Page </a> </li>
  <li> <a href ="browse.php"> Activities </a> </li>
  <li> <a href ="burro.php"> Training </a> </li>
  <li> <a href ="add_training.php"> Add New Training </a> </li>
</ul>
```

Appendix B

Table	Action	Rows	Type	Collation	Size	Overhead
☐ Activities	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb3_bin	16.0 KiB	-
☐ Training	Browse Structure Search Insert Empty Drop	7	InnoDB	utf8mb3_bin	16.0 KiB	-
2 tables	Sum	11	InnoDB	utf8mb3_bin	32.0 KiB	0 B

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	ID	Name	Description	Benefits	Price
☐ Edit Copy Delete	1	Fitness	General fitness activities encompass a wide range...	Improves cardiovascular health; supports weight m...	From £30 per hour
☐ Edit Copy Delete	2	Strength Training	Strength training involves using resistance to bu...	Builds muscle mass; improves bone density; boosts...	From £50 per 45 minutes
☐ Edit Copy Delete	3	Yoga	Mind-body practice that combines physical posture...	Improves flexibility and balance; reduces stress	From 35 per 45 minutes
☐ Edit Copy Delete	4	Pilates	Low-impact exercise that focuses on strengthening...	Strengthens core muscles; improves posture; incre...	From 30 per 45 minutes

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	ID	Name	E-mail	Location	Certifications	Years	Specialization
☐ Edit Copy Delete	1	Mary Brown	mary@may.com	NW3 only	Level 3	3	Fitness, Yoga
☐ Edit Copy Delete	2	James White	james@james.com	SW1 and online	Level 3	5	Fitness, Strength Training
☐ Edit Copy Delete	3	Ann Blue	ann@ann.com	Online only	ISSA	5	Pilates, Strength Training
☐ Edit Copy Delete	4	Peter Red	peter@peter.com	NW2, NW2, NW3	Level 3	4	Pilates